

Module 0 — Environment Setup for Agentic AI (Pre-Module A)

Status:  PROPOSED FOR LOCK

Audience: Absolute beginners

Duration: 60–90 minutes (one-time)

Positioning:

This module exists to **remove friction**, not to teach programming.

Purpose of Module 0

- Ensure every learner can **run Python code locally**
 - Standardize tools to avoid support chaos
 - Prevent installation issues from blocking learning
 - Build *confidence* before learning begins
-

What This Module Covers (Only What's Necessary)

0.1 What We're Installing (High-Level)

- Python → runs our code
- VS Code → writes and runs code
- Terminal → executes commands

No internals. No theory. Just purpose.

0.2 Installing Python

- Supported versions (explicitly stated)
- How to verify installation (`python --version`)
- PATH explanation (conceptual, not deep)

⚠ OS-specific walkthroughs:

- Windows
 - macOS
 - (Linux optional / minimal)
-

0.3 Installing VS Code

- What VS Code is (and is not)
 - Required extensions:
 - Python
 - Pylance (for typing later)
 - Opening folders vs files
-

0.4 First Successful Run (Critical Moment)

- Create a folder
- Create `hello.py`
- Run it from:

- VS Code
- Terminal

Psychological milestone:
“I ran code on my machine.”

0.5 Virtual Environments (Concept Only)

- Why environments exist
- Create & activate one (copy-paste friendly)
- When we'll actually use it (later modules)

No deep explanation yet.

0.6 Installing Packages (Preview Only)

- `pip install`
- Why dependencies matter for AI projects
- No third-party libraries yet

Module A — Python for Agentic AI

Status:  FINAL (Ready to Lock)

Audience: Absolute beginners

Role: Build reliable thinking primitives for Agentic AI systems

Design principle:

Python is taught here **as an agent control language**, not as general software engineering.

Module A — Learning Outcomes

By the end of Module A, learners will:

- Think in **state, flow, and constraints**
 - Read and write Python code confidently
 - Build predictable logic around probabilistic LLMs
 - Be fully prepared for:
 - Prompt construction
 - Tool invocation
 - Agent workflows
 - Typed state graphs (LangGraph)
-

A1. Programming Mindset for AI Systems

- What programming actually is (instructions → state changes)
- Deterministic code vs probabilistic AI behavior

- Why agents need *strict control* around LLMs
- Python's role in the Agentic AI stack

Agent framing:

"Code controls the system. Models provide suggestions."

A2. Running Python (Zero-Friction Setup)

- Python interpreter vs scripts
- Running `.py` files (CLI + editor)
- Virtual environments (why isolation matters)
- Installing packages with `pip`

Outcome: Learners can run, modify, and re-run code safely.

A3. Variables & Data as Agent Memory

- Variables as named memory slots
- Numbers, strings, booleans
- Reassignment and mutation
- Printing and inspecting values
- **Arithmetic and logical operators used for state changes and decisions**

Agent framing:

"This is how an agent remembers information and updates its internal state."

A4. Strings Deep Dive (LLM-Critical)

- Text as first-class data
- String concatenation & f-strings
- Multiline strings for prompts
- Formatting, clarity, and whitespace discipline
- **Essential string manipulation used in prompt construction**

Note: This section is intentionally deeper than standard Python courses due to its importance in LLM systems.

A5. Lists, Dictionaries, Sets & Tuples — Core Agent Data Structures

- **Lists** → ordered sequences (plans, messages, steps)
- **Dictionaries** → structured state (agent memory)
- **Sets** → uniqueness & constraints (prevent repeated actions)
- **Tuples** → fixed, trusted data (guarantees)

Decision rule taught explicitly:

Need	Use
Ordered flow	List
Named state	Dictionary
No duplicates	Set
Immutable data	Tuple

A6. Control Flow (Agent Decision Logic)

- `if / elif / else`
- Boolean logic
- Rule-based checks before LLM calls
- Guardrails vs delegation

Key insight:

Not every decision should be handled by an LLM.

A7. Loops (Iteration & Evaluation)

- `for` loops over sequences
- Safe iteration patterns
- Avoiding infinite loops
- Evaluating multiple candidate outputs

Agent framing:

“This is how agents explore and evaluate options.”

A8. Functions (Agent Capabilities)

- Functions as reusable abilities
- Inputs → outputs → side effects
- Naming functions like agent skills

- Composition over complexity

Mental model:

“Agents orchestrate small, reliable functions.”

A9. Errors, Safety & Debugging

- Syntax vs runtime errors
- Reading tracebacks calmly
- Defensive checks
- Graceful failure patterns

Why this matters:

Agents must fail **safely and visibly**, not silently.

A10. Minimal File Structure for AI Projects

- Single-file scripts vs multi-file projects
 - Why agents naturally grow into folders
 - Imports (light introduction)
 - Preparing for LangChain & LangGraph
-

Module A+ — Type Annotations for Agent State (Pre-LangGraph Bridge)

Status:  FORMALLY LOCKED

Purpose: Prepare learners for LangGraph without overwhelming them

Framing:

Type annotations are introduced as **state safety and clarity tools**, not Python theory.

A+.1 Why Types Matter in Agent Systems

- Agent state crosses steps and nodes
 - Types act as contracts for agent memory
 - Prevent ambiguity in workflows
-

A+.2 Basic Type Annotations

```
name: str
count: int
score: float
active: bool
```

A+.3 Typed Lists & Dictionaries

```
steps: list[str]
metadata: dict[str, str]
```

A+.4 **Optional** — Incomplete State Is Normal

```
from typing import Optional
summary: Optional[str]
```

A+.5 Union — Multiple Valid Outcomes

```
from typing import Union
result: Union[str, dict]
```

A+.6 TypedDict — Agent State Blueprint (Critical)

```
from typing import TypedDict

class AgentState(TypedDict):
    question: str
    context: list[str]
    answer: Optional[str]
```

Mental model:

“This defines the shape of my agent’s brain.”

Module B — LLMs & Prompt Engineering

Status: 🟡 Draft for Final Lock

Audience: Absolute beginners (no ML background assumed)

Role: Build correct mental models for working with LLMs *before* agents, tools, or frameworks

Design principle:

Prompting is **interface design**, not magic and not trial-and-error.

Module B — Learning Outcomes

By the end of Module B, learners will:

- Understand what LLMs *are* and *are not*
 - Predict common failure modes (hallucination, verbosity, inconsistency)
 - Write clear, structured prompts
 - Separate **instruction**, **context**, and **output format**
 - Safely integrate LLMs into Python-controlled workflows
-

B1. What Large Language Models Actually Are

- Probability-driven text generators (conceptual)
- Tokens vs words (high level)
- Training vs inference (no math)
- Why confidence $\neq$ correctness

Correction taught early:

“LLMs predict text; they do not reason or know.”

B2. What LLMs Are Good At vs Bad At

Good at

- Language understanding
- Pattern recognition
- Summarization
- Explanation

Bad at

- Long-term memory
- Deterministic consistency
- Math without structure
- Self-verification

Agent framing:

LLMs are advisors, not decision-makers.

B3. Determinism vs Variability

- Why identical prompts yield different outputs
- Temperature (conceptual, not tuning)
- Why agents require deterministic code around probabilistic models

Mental model:

Code = guardrails
Model = suggestion engine

B4. Prompt Anatomy (Foundational Section)

Prompts are taught as **structured inputs**, not text blobs.

Components

1. Role (expertise boundary)
2. Instruction (task)
3. Context (supporting information)
4. Constraints (rules and limits)
5. Output format (structure)

This structure is used consistently throughout the course.

B5. Instruction vs Context (Major Failure Point)

- Why “I gave it the info” often fails
 - Context is input, not intent
 - Explicit vs implicit instructions
 - Structuring context blocks clearly
-

B6. Output Control & Formatting

- Asking for structured outputs

- JSON-like shapes (conceptual only)
 - Why formatting is non-negotiable for code consumption
 - Failure modes caused by loose formatting
-

B7. Prompt Iteration (Without Guessing)

- Diagnosing *why* a prompt failed
- Tightening instructions vs adding examples
- When examples help — and when they hurt
- Avoiding prompt bloat

Key insight:

Better prompts are clearer, not longer.

B8. Safety, Hallucinations & Trust Boundaries

- Why hallucinations happen
 - Why “don’t hallucinate” doesn’t work
 - Practical mitigation strategies:
 - Scope restriction
 - Explicit uncertainty
 - Evidence requests (conceptual)
-

B9. Prompting vs Programming (Critical Boundary)

- What logic must stay in code
- What can be delegated to LLMs
- Why agents exist
- The cost of over-prompting

Bridge insight:

Prompting without code creates fragile systems.

B10. Prompt Engineering Patterns (Explicit Naming)

(No new concepts — explicit naming only)

Learners are shown that the techniques they already used have names:

- **Zero-shot prompting** — clear instructions with no examples
- **Few-shot prompting (In-context learning)** — adding minimal examples to correct failures
- **Role / persona assignment** — setting expertise boundaries (not personalities)
- **Contextual prompting** — providing structured supporting information
- **Chain-of-thought prompting** — requesting reasoning (*used cautiously; often internalized later*)

Positioning statement:

These are not tricks — they are applications of prompt structure.

Module C — Embeddings & Semantic Search

Status: 🟡 Draft for Final Lock

Audience: Beginners (no ML or math background)

Role: Teach machines to *find relevant information* instead of guessing

Design principle:

Embeddings are not “AI magic” — they are **meaning encoders** that enable reliable retrieval.

Module C — Learning Outcomes

By the end of Module C, learners will:

- Understand what embeddings represent (and what they don't)
 - Explain semantic similarity in plain language
 - Build a simple semantic search pipeline
 - Know when embeddings outperform keyword search
 - Be fully prepared for RAG ingestion and retrieval flows
-

C1. The Core Problem: Why LLMs Guess

- LLMs generate text based on likelihood
- They do not “look things up”
- Why guessing is unacceptable for enterprise systems

Bridge insight:

If correctness matters, retrieval must precede reasoning.

C2. What Embeddings Actually Are (Beginner-Safe)

- Text → numbers (vectors) that represent meaning
- Similar meaning → closer vectors
- No math, no linear algebra

Mental model:

Embeddings are coordinates in a “meaning space”.

C3. Semantic Similarity vs Keyword Search

- Why keyword search fails with:
 - Synonyms
 - Paraphrasing
 - Natural language questions
- How semantic search fixes this

Examples discussed conceptually:

- “refund policy” vs “money back rules”
 - “heart attack” vs “myocardial infarction”
-

C4. Embeddings Lifecycle (End-to-End View)

Learners see the full flow **before** code details.

1. Prepare text (chunking comes later)
2. Convert text to embeddings
3. Store embeddings
4. Convert query to embedding
5. Compare similarity
6. Retrieve top matches

This lifecycle is reused unchanged in RAG and agents.

C5. Similarity Search (Conceptual, Not Mathematical)

- What “closest” means
- Why cosine similarity is commonly used (intuition only)
- Ranking results by relevance

Key insight:

Embeddings don't answer questions — they *select evidence*.

C6. Building a Minimal Semantic Search (Hands-On)

- Small text dataset
- Generate embeddings
- Query with natural language
- Retrieve most relevant chunks

Important constraint:

- In-memory only
- No vector databases yet

This keeps focus on **concepts, not tools**.

C7. What Embeddings Are Good At vs Bad At

Good at

- Finding relevant information
- Handling paraphrases
- Semantic clustering

Bad at

- Exact facts (IDs, numbers)
- Logical reasoning
- Replacing databases

Critical warning:

Embeddings retrieve relevance, not truth.

C8. Common Failure Modes

- Poor chunk boundaries
- Too much or too little text
- Garbage in → garbage retrieval

- Over-trusting similarity scores

Failures are framed as **design mistakes**, not model flaws.

C9. Where Embeddings Fit in Agentic Systems

- Retrieval before reasoning
- Evidence feeding prompts
- Reducing hallucinations
- Making agents grounded and auditable

Mental model:

Embeddings are the agent's *search engine*, not its brain.

Module D — RAG (Retrieval → Reasoning → Grounded Output)

Status: 🟡 Draft for Final Lock

Audience: Beginners (concept-ready)

Role: Teach how to build **grounded, reliable LLM systems** using retrieval

Design principle:

RAG is not a feature — it is a **system architecture pattern**.

Module D — Learning Outcomes

By the end of Module D, learners will:

- Understand *why* RAG exists and what problems it solves
 - Design a full RAG pipeline end-to-end
 - Use vector databases appropriately (concept + practice)
 - Diagnose common RAG failure modes
 - Be ready to implement RAG using LangChain abstractions
-

D1. Why RAG Exists (Reframing the Problem)

- LLMs do not “know” your data
- Prompt stuffing does not scale
- Hallucinations are often *missing retrieval problems*

Core insight:

If the model didn't see the data, it will guess.

D2. RAG Architecture — The Big Picture

RAG =

1. **Ingestion** — prepare and store knowledge
2. **Retrieval** — find relevant evidence
3. **Reasoning** — generate grounded output

This framing stays consistent across the rest of the bootcamp.

D3. Ingestion Phase (Garbage In = Garbage Retrieval)

- What “ingestion” actually means
- Typical data sources (documents, FAQs, notes)
- Why naïve ingestion fails
- Preview of chunking (conceptual only)

Key warning:

Bad ingestion cannot be fixed by better prompts.

D4. Vector Databases — First Formal Introduction (UPDATED)

This is the **correct and only place** where vector databases are *introduced conceptually*.

What a Vector Database Provides

- Persistent storage of embeddings
- Fast similarity search
- Retrieval at scale

Mental model:

Embeddings are the data. Vector databases are the retrieval engine.

Conceptual Comparison (Beginner-Safe)

Tool	Role	When to Use
FAISS	Local vector index	Learning, experiments, offline use
Chroma	Developer-friendly local DB	Prototyping, small-to-medium RAG apps
Pinecone e	Managed vector DB	Production, scale, reliability

How this is framed to learners:

- FAISS → *library*
- Chroma → *local database*
- Pinecone → *managed service*

No infra tuning. No ops discussion. No vendor lock-in.

Lifecycle (Common Across All Vector DBs)

1. Write embeddings
2. Store vectors
3. Query by similarity

4. Retrieve top matches

The backend changes — the pattern does not.

D5. Retrieval Phase (Evidence Selection, Not Answers)

- Query → embedding → similarity search
- Top-k retrieval (conceptual)
- Why retrieval ≠ correctness

Critical reframing:

Retrieval selects *evidence*, not truth.

D6. Reasoning Phase (LLM with Guardrails)

- Passing retrieved context into prompts
- Separating:
 - User question
 - Retrieved evidence
 - Instructions
- Why reasoning must be constrained

Prompt anatomy from Module B is reused *unchanged*.

D7. End-to-End RAG Flow (Conceptual Walkthrough)

1. User asks a question
2. System retrieves relevant chunks
3. Prompt is constructed with evidence
4. LLM produces grounded output

This is the **mental model they will later implement**.

D8. Common RAG Failure Modes

- Retrieval returns irrelevant chunks
- Context overload
- Missing critical information
- Over-trusting similarity scores

Failures are framed as **design errors**, not model flaws.

D9. When RAG Is the Wrong Tool

- Dynamic data (live APIs)
- Precise calculations
- Transactional systems

Boundary reminder:

RAG improves grounding, not logic.

Module E — LangChain (LLM Application Orchestration)

Status: 🟡 Draft for Final Lock

Audience: Beginners (architecture-ready)

Role: Teach LangChain as **orchestration glue**, not as “magic abstractions”

Design principle:

LangChain does not add intelligence — it **organizes execution**.

Module E — Learning Outcomes

By the end of Module E, learners will:

- Understand what LangChain is (and is not)
 - Use LangChain to orchestrate prompts, LLMs, retrievers, and tools
 - Build predictable execution flows
 - Recognize when chains are sufficient vs when agents are required
 - Be fully prepared for LangGraph
-

E1. Why LangChain Exists

- Reduces repetitive glue code
- Standardizes common LLM application patterns
- Optional, not mandatory

Boundary set early:

LangChain simplifies structure — it does not replace system design.

E2. Core LangChain Mental Model

LangChain is taught as **layers**, not APIs:

- **Models** — LLM interfaces
- **Prompts** — structured inputs
- **Chains** — execution flow
- **Retrievers** — access to knowledge
- **Tools** — callable capabilities

Mental model:

“LangChain wires components together.”

E3. Prompts in LangChain

- Translating prompt anatomy (Module B) into LangChain
- Prompt templates and variables
- Reuse and consistency

Reminder:

A bad prompt is still bad inside LangChain.

E4. LLM Invocation & Output Handling

- Calling LLMs via LangChain

- Handling responses safely
- Guarding output shape (conceptual)

No tuning deep dive yet.

E5. Chains — Execution Patterns (NEW & IMPORTANT)

Chains are introduced as **execution patterns**, not just sequences.

E5.1 Sequential Chains

- Steps run in a fixed order
- Output of one step feeds the next

Use cases:

- Retrieve → summarize
- Draft → refine
- Extract → format

Mental model:

“Do this, then that.”

E5.2 Conditional Chains

- Branching based on a condition
- Different paths for different inputs or outcomes

Use cases:

- If confidence is low → retrieve more context
- If input is short → ask clarifying question
- If validation fails → retry or fallback

Mental model:

“Decide, then act.”

E5.3 Parallel Chains

- Multiple chains run independently
- Results are merged or compared

Use cases:

- Generate multiple answers
- Compare summaries
- Run analysis and explanation simultaneously

Mental model:

“Explore options at the same time.”

Critical Boundary (Explicitly Taught)

Chains are **structured flows**, not agents.

They:

- Do not reason about goals
- Do not adapt strategy

- Do not maintain long-lived state

This sets up LangGraph *cleanly*.

E6. Retrieval with LangChain (RAG Implementation)

- Implementing RAG from Module D
- Using retrievers with:
 - FAISS
 - Chroma
 - Pinecone
- Swapping vector backends without logic changes

Key takeaway:

Retrieval logic stays constant; storage is pluggable.

E7. Tools in LangChain

- What a tool is
- When tools beat prompting
- Input/output discipline

Bridge to agents:

Tools are how systems act, not think.

E8. Common LangChain Failure Modes

- Over-chaining
- Hidden complexity
- Debugging pain
- Treating LangChain as “the brain”

Failures framed as **design mistakes**, not framework issues.

E9. When *Not* to Use LangChain

- Simple scripts
- One-off prompts
- Tight latency requirements

Principle reinforced:

Use LangChain when it reduces complexity.

Module F — Agents in LLM Systems (Mental Models + Examples)

Status: 🟡 Draft for Final Lock

Audience: Beginners (implementation-aware)

Role: Teach *what an agent actually is* — and what it is not

Design principle:

An agent is not an LLM.

An agent is a **system that decides what to do next**.

Module F — Learning Outcomes

By the end of Module F, learners will:

- Understand what qualifies as an agent (and what doesn't)
 - Distinguish agents from chains, tools, and prompts
 - Reason about agent autonomy and control
 - Recognize common agent failure modes
 - Be mentally prepared for stateful agent workflows (LangGraph)
-

F1. Why “Agents” Exist

- Limits of chains:
 - Fixed paths
 - No adaptability
- Real systems must:
 - Decide

- Retry
- Change strategy
- Stop

Core insight:

Agents exist when *flow cannot be fully predicted upfront*.

F2. What an Agent Is (Precise Definition)

An agent is a system that:

1. Observes state
2. Chooses an action
3. Executes the action
4. Updates state
5. Decides what to do next

Key correction:

LLM $\neq$ agent

LLM = reasoning component inside an agent

F3. What Is *Not* an Agent (Critical Clarity)

Explicitly contrasted to avoid confusion.

Pattern	Why It's Not an Agent
Prompt	No decision loop
Chain	Fixed execution

Tool call Action, not control
RAG pipeline Retrieval, not autonomy

This prevents 90% of beginner misunderstandings.

F4. Core Components of an Agent

Agents are decomposed into **clear parts**:

- **State** — what the agent knows
- **Policy** — how decisions are made
- **Actions** — what the agent can do
- **Environment** — external systems
- **Loop** — observe → act → update

Mental model:

“An agent is a loop with memory and choice.”

F5. Levels of Autonomy (Important Safety Gradient)

Level 0 — Scripted

- Fully deterministic
- No LLM decisions

Level 1 — Assisted

- LLM suggests
- Code decides

Level 2 — Semi-Autonomous

- LLM selects from constrained actions
- Guardrails enforced

Level 3 — Autonomous (Cautious)

- LLM plans and acts
- Requires strong controls

Teaching principle:

We climb autonomy levels slowly and intentionally.

F6. Simple Agent Examples (Conceptual, Not Framework-Specific)

Examples discussed without LangGraph or APIs:

- Question-answering agent
- Research assistant agent
- Validation agent
- Retry-on-failure agent

Focus is on:

- Decision logic
 - State transitions
 - Stop conditions
-

F7. Agent Failure Modes (Must Be Taught Early)

- Infinite loops
- Over-reasoning
- Tool thrashing
- Hallucinated decisions
- Cost explosions

Failures are framed as **design problems**, not LLM problems.

F8. Guardrails & Stop Conditions

- Explicit stop criteria
- Max steps
- Confidence thresholds
- Human-in-the-loop (conceptual)

Key principle:

A safe agent knows when to stop.

F9. Agents vs Workflows (Bridge to LangGraph)

- Workflows:
 - Predefined
 - Predictable

- Agents:
 - Adaptive
 - State-driven

Bridge insight:

LangGraph exists to *control agent workflows safely*.

(No LangGraph mechanics yet — mental prep only.)

Module G — LangGraph (State + Controlled Agent Workflows)

Status: 🟡 Draft for Final Lock

Audience: Beginners (agent-ready)

Role: Teach **stateful, governed agent workflows** using LangGraph

Design principle:

LangGraph exists to make agent systems **explicit, inspectable, and safe**.

Module G — Learning Outcomes

By the end of Module G, learners will:

- Understand why LangGraph exists (and why chains/agents alone are insufficient)
 - Model agent state explicitly
 - Build deterministic, state-driven workflows
 - Control agent loops, branches, and stop conditions
 - Design agents that are debuggable and production-ready
-

G1. Why LangGraph Exists

- Problems with:
 - Free-form agents
 - Hidden control flow
 - Infinite loops

- Why enterprise systems require:
 - Explicit state
 - Visible transitions
 - Controlled autonomy

Core insight:

LangGraph turns “agent behavior” into **a graph you can reason about**.

G2. Core LangGraph Mental Model

LangGraph is introduced as **three ideas**, not APIs:

1. **State** — a typed object that represents agent memory
2. **Nodes** — pure functions that read/update state
3. **Edges** — rules that decide what runs next

Mental model:

“LangGraph is a state machine for LLM systems.”

G3. Defining State (Critical Section)

Directly builds on **Module A+ (TypedDict)**.

- State as a single source of truth
- Typed state schemas
- Why explicit state beats hidden memory

Example concepts (no code deep dive yet):

- Input fields
- Intermediate results
- Decisions
- Final output

Reinforced principle:
If it's not in state, it doesn't exist.

G4. Nodes — Agent Capabilities as Functions

- Nodes as deterministic functions
- Reading from state
- Writing back to state
- No hidden side effects

Connection to earlier modules:

- Nodes = functions (Module A)
 - Nodes may call:
 - LLMs
 - Tools
 - Retrievers
-

G5. Edges — Control Flow & Decisions

- Sequential edges

- Conditional edges
- Loop edges (with guardrails)

Key teaching point:

All agent “thinking” is just **state-driven branching**.

G6. Loops, Retries & Stop Conditions

This is where LangGraph shines.

- Controlled iteration
- Retry on failure
- Max step limits
- Explicit termination

Safety principle:

Every loop must have a stop condition.

G7. Visualizing & Debugging Agent Workflows

- Why visibility matters
- Inspecting state at each step
- Understanding why an agent made a decision

Enterprise framing:

If you can't explain it, you can't ship it.

G8. Implementing RAG Inside LangGraph

- Retrieval as a node
- Reasoning as a node
- State carries evidence explicitly
- Prompt anatomy reused unchanged

Key takeaway:

LangGraph does not change RAG — it **controls** it.

G9. When to Use LangGraph vs LangChain

Clear decision boundary:

Use Case	Tool
Simple orchestration	LangChain
Linear flows	Chains
Stateful agents	LangGraph
Governed autonomy	LangGraph

Module H — MCP (Context Boundaries + Governance)

Status: 🟡 Draft for Final Lock

Audience: Beginners → Professionals

Role: Teach how to **control what agents see, do, and decide** — safely and audibly

Design principle:

MCP is not a tool — it is a **governance mindset** for Agentic AI systems.

Module H — Learning Outcomes

By the end of Module H, learners will:

- Understand why context boundaries are critical in agent systems
 - Design agents with explicit permissions and scope
 - Prevent context leakage and overreach
 - Apply governance principles to agent workflows
 - Reason about trust, safety, and accountability in Agentic AI
-

H1. The Core Problem: Unbounded Context

- LLMs eagerly use *any* context they are given
- Bigger context ≠ better behavior
- Why uncontrolled context leads to:
 - Hallucinations

- Data leakage
- Unsafe decisions

Core insight:

An agent without boundaries is a liability.

H2. What MCP Means (Conceptual Definition)

MCP = Model Context Protocol (conceptual)

Taught as:

- A way of **structuring, limiting, and governing context**
- Not a specific vendor or SDK
- A set of design rules for agent systems

Mental model:

“MCP defines *what the agent is allowed to know and use.*”

H3. Types of Context in Agent Systems

Learners learn to distinguish context types clearly:

- **Instruction context** — rules and goals
- **Operational context** — current state
- **Knowledge context** — retrieved data
- **User context** — inputs and preferences
- **System context** — limits and policies

Key principle:

Mixing context types causes unpredictable behavior.

H4. Context Boundaries (The Heart of MCP)

- Why agents should *not* see everything
- Scope-limited context per node or step
- Passing only what is required

Connection to LangGraph:

State + edges naturally enforce context boundaries.

H5. Permissions & Capability Control

- Not every agent can do everything
- Explicitly allowed actions vs forbidden ones
- Tool access as permissions

Mental model:

Tools are permissions, not conveniences.

H6. Governance Rules & Guardrails

- Hard rules (must / must not)
- Soft rules (guidelines)
- Enforced vs advisory constraints

Examples discussed conceptually:

- Data access restrictions
 - Action limits
 - Escalation conditions
-

H7. Auditability & Explainability

- Why enterprises require explanations
- Logging decisions and state transitions
- Being able to answer:
 - “Why did the agent do this?”
 - “What did it see?”

Principle:

If you can't audit it, you can't trust it.

H8. Common MCP Failure Modes

- Context overload
- Context leakage between steps
- Implicit permissions
- Over-trusting the LLM

Failures are framed as **design issues**, not model failures.

H9. MCP in Practice (Conceptual Walkthrough)

A full mental walkthrough:

1. User request arrives
2. Context is scoped
3. Permissions are checked
4. Agent acts within bounds
5. Decisions are logged
6. System stops safely

No code — pure system thinking.

Module I — APIs, Tools, Streamlit, Git/GitHub

Status: 🟡 Draft for Final Lock

Audience: Beginners → Practitioner-ready

Role: Teach learners how to **package, expose, interact with, and version Agentic AI systems**

Design principle:

An AI system that cannot be run, tested, or shared **does not exist**.

Module I — Learning Outcomes

By the end of Module I, learners will:

- Understand how agents interact with external systems via APIs and tools
 - Expose AI functionality through simple user interfaces
 - Package agent workflows for reuse
 - Use Git/GitHub to version, track, and share work
 - Deliver a complete, runnable Agentic AI project
-

I1. APIs — How Agentic Systems Talk to the World

- What an API is (conceptual, no networking theory)
- Why APIs are the backbone of AI systems
- Request → response mental model
- Where APIs fit in agent workflows

Agent framing:

APIs are how agents *observe* and *act* outside themselves.

12. Using APIs Inside Agent Systems

- Calling external services (conceptual examples)
- Handling responses safely
- Failures, timeouts, and fallbacks (conceptual)

Boundary taught clearly:

Agents *use* APIs — they do not depend on them blindly.

13. Tools — Turning Code into Agent Capabilities

- Tools as callable functions
- Input/output discipline
- When tools beat prompting
- Mapping tools to permissions (tie-back to MCP)

Bridge insight:

Tools define what an agent is *allowed* to do.

14. Streamlit — Simple Interfaces for AI Systems

- Why UI matters (even for internal tools)
- Streamlit as a rapid interface layer

- Inputs → outputs → state

What is taught

- Running a Streamlit app
- Wiring inputs to agent workflows
- Displaying results clearly

No UI design theory. No frontend frameworks.

15. Integrating LangChain / LangGraph with Streamlit

- Triggering chains and graphs from UI actions
- Displaying intermediate and final outputs
- Keeping UI separate from core logic

Best practice emphasized:

UI calls the system — the system does not live in the UI.

16. Git & GitHub — Versioning Agentic AI Projects

- Why version control matters for AI
- Git as a history of decisions
- GitHub as collaboration & sharing

What is taught

- Repository basics

- Commits with intent
- Tracking experiments vs stable code

No advanced Git workflows.

17. Structuring an Agentic AI Project (End-to-End)

Learners see a **clean, minimal structure**:

- Core logic (agents, graphs, tools)
- Retrieval & data
- UI layer
- Config & environment

Key principle:

Separation of concerns prevents chaos.

18. Common Delivery Failure Modes

- Logic buried in UI code
- No version history
- Hard-coded secrets
- Unreproducible demos

Failures are framed as **engineering hygiene issues**, not AI problems.

19. From Prototype to Shareable System

- What “done” looks like
- Demo-readiness checklist
- Explaining system behavior to others

Professional framing:

If someone else can't run it, it's not finished.

Module J — Advanced RAG (Design Patterns & Reliability)

Status: 🟡 Draft for Final Lock

Audience: Agent-ready learners

Role: Move from *basic RAG* to **reliable, production-grade retrieval systems**

Design principle:

Advanced RAG is about **reducing failure modes**, not adding complexity.

Module J — Learning Outcomes

By the end of this module, learners will:

- Design RAG systems that scale and remain reliable
 - Improve retrieval precision and relevance
 - Handle ambiguity and missing information
 - Choose the right RAG pattern for the problem
 - Prepare RAG systems for agent control
-

J1. Why Basic RAG Fails at Scale

- Irrelevant retrieval
- Context overload
- Partial answers
- False confidence

Key insight:

Most RAG failures are *retrieval design problems*, not LLM issues.

J2. Chunking Strategies (Now Taught Properly)

- Fixed vs semantic chunking
 - Chunk size trade-offs
 - Overlap strategies
 - Why chunking decisions matter more than model choice
-

J3. Metadata-Aware Retrieval

- Attaching metadata during ingestion
- Filtering before similarity search
- Using metadata to reduce noise

Mental model:

Retrieve *less*, but retrieve *better*.

J4. Hybrid Search (Semantic + Keyword)

- Why pure embeddings sometimes fail
 - Combining keyword and vector search (conceptual)
 - When hybrid search is necessary
-

J5. Multi-Stage Retrieval

- Retrieve → refine → retrieve
- Coarse vs fine search
- Reranking (conceptual)

Agent bridge:

Retrieval itself can become a decision process.

J6. Handling Ambiguity & Missing Context

- Detecting weak retrieval signals
 - Asking clarifying questions
 - Fallback strategies
-

J7. Evaluating RAG Quality

- What “good retrieval” means
 - Qualitative evaluation
 - Human-in-the-loop review (conceptual)
-

What We Still Do NOT Teach

- ✗ Custom embedding models
- ✗ Vector DB internals
- ✗ Offline evaluation pipelines

These belong in ML-heavy tracks.

Final Outcome for Module J

Learners can:

- Design robust RAG systems
- Explain RAG decisions to stakeholders
- Prepare RAG pipelines for agent orchestration

Module K — Multi-Agent Systems & Orchestration

Status: 🟡 Draft for Final Lock

Audience: Advanced learners (agent-ready)

Role: Teach how to build **coordinated, governed multi-agent systems** — not uncontrolled agent swarms

Design principle:

Multi-agent systems only work when **orchestration is explicit**.

Module K — Learning Outcomes

By the end of Module K, learners will:

- Understand *when* multi-agent systems are appropriate
 - Design agents with **clear, bounded responsibilities**
 - Implement a **central orchestration layer**
 - Coordinate agents safely using LangGraph
 - Prevent common multi-agent failure modes
 - Explain and defend orchestration decisions
-

K0. Orchestration First (Why This Module Exists)

Before introducing multiple agents, learners are taught **what usually goes wrong**:

- Agents conflict with each other
- Tasks are delegated endlessly

- Context leaks between agents
- Costs spiral
- No one can explain *why* a decision was made

Core correction:

Multi-agent systems are **not social systems**.
They are **distributed workflows with a controller**.

K1. What “Multi-Agent” Actually Means (Correct Definition)

A multi-agent system is:

- Multiple **specialized agents**
- Coordinated by a **single orchestration layer**
- Sharing **explicit, governed state**
- Executing toward a **single system goal**

Key clarification:

- Agents do *not* coordinate themselves
 - Agents do *not* negotiate freely
 - Agents do *not* control execution order
-

K2. The Orchestration Layer (Core Concept)

The orchestration layer is the **control plane**.

Responsibilities

- Decide **which agent acts next**
- Assign **specific tasks**
- Control **turn-taking**
- Enforce **stop conditions**
- Manage **shared vs isolated state**
- Apply **governance rules (MCP)**

Mental model:

Orchestration controls *flow*.

Agents perform *work*.

K3. Agents as Specialists (Bounded Roles)

Agents are designed as **single-responsibility components**.

Common agent roles

- **Planner agent** — decomposes goals
- **Retrieval agent** — gathers evidence
- **Analysis agent** — reasons over inputs
- **Validator agent** — checks quality or safety
- **Execution agent** — performs actions

Explicit rule:

An agent must be able to explain *exactly what it is responsible for*.

No personalities. No “general intelligence” agents.

K4. State Design in Multi-Agent Systems

Builds directly on **Module A+ (TypedDict)** and **Module G (LangGraph state)**.

State types

- **Global state** — shared system context
- **Agent-local state** — temporary, scoped
- **Orchestration state** — decisions, progress, limits

Critical principle:

If state boundaries are unclear, agent behavior becomes unpredictable.

K5. Orchestration Patterns (Explicitly Taught)

Learners are taught **named, reusable orchestration patterns**.

K5.1 Supervisor / Worker Pattern

- Orchestrator assigns tasks
- Workers execute
- Results return to orchestrator

Use when:

- Tasks are decomposable
 - Control must remain centralized
-

K5.2 Plan → Execute → Validate

- Planner proposes steps
- Executor performs
- Validator checks outcome

Use when:

- Quality matters
 - Errors are costly
-

K5.3 Fan-Out → Fan-In

- Parallel agent execution
- Results merged or compared

Use when:

- Exploration is required
 - Time can be traded for quality
-

K5.4 Escalation & Fallback

- If confidence is low → escalate
- If validation fails → retry or stop

Use when:

- Safety or correctness is critical
-

K6. LangGraph as the Orchestrator

LangGraph is positioned **clearly and correctly**:

- **Nodes** → agent invocations
- **Edges** → orchestration rules
- **State** → shared contract

Key teaching point:

LangGraph is not “agent magic” — it is **explicit orchestration logic**.

No free-running agents.

No hidden loops.

No invisible decisions.

K7. Governance in Multi-Agent Systems (MCP Applied)

Direct integration with **Module H (MCP)**.

Governance enforced at orchestration level

- Context scoping per agent
- Tool permissions per agent
- Budget & step limits
- Explicit stop conditions
- Audit logs

Critical insight:

Governance belongs in orchestration — not inside agents.

K8. Multi-Agent Failure Modes (Orchestration View)

Failures are analyzed as **design flaws**, not LLM issues.

- Infinite delegation loops
- Conflicting agent outputs
- Orchestrator blind spots
- Over-parallelization
- Thrashing between agents

Learners are taught to **trace failures back to orchestration decisions**.

K9. When NOT to Use Multi-Agent Systems

Multi-agent systems are **not always the answer**.

Do **not** use them when:

- The task is linear
- The goal is clear
- Latency is critical
- A single agent suffices

Rule taught explicitly:

If orchestration adds more complexity than value — don't use it.

Module L — Capstone Project (Enterprise-Grade Agentic AI System)

Status:  UPDATED & READY TO LOCK

Role: Prove end-to-end capability in **design, control, governance, and delivery**

Capstone principle:

The capstone is not a demo — it is a **defensible system architecture**.

Capstone Objective

Build a **complete Agentic AI system** that demonstrates:

- Grounded reasoning (RAG)
 - Controlled autonomy (LangGraph)
 - Governance & context boundaries (MCP)
 - Delivery (API/UI + version control)
 - **Orchestration-first design**
-

Mandatory Architecture Requirements (All Projects)

Every capstone **must include**:

1. **Explicit State Definition**
 - Typed state (`TypedDict` or equivalent)
 - State passed explicitly between steps
 - No hidden or implicit memory

2. **Controlled Workflow**

- LangGraph used for:
 - Flow control
 - Branching
 - Stop conditions
- No free-running loops

3. **Grounded Reasoning**

- RAG or equivalent retrieval mechanism
- Clear separation of:
 - Question
 - Evidence
 - Reasoning

4. **Governance (MCP Applied)**

- Context boundaries enforced
- Tool permissions scoped
- Explicit limits (steps, retries, cost)

5. **Delivery Layer**

- Streamlit UI **or** API interface
- Runnable by another person
- No hard-coded secrets

6. **Version Control**

- GitHub repository

- Meaningful commit history
- Clear project structure

Single-Agent vs Multi-Agent Capstones (Explicit Rules)

✓ Single-Agent Capstone

Allowed when:

- The problem is linear or moderately complex
- One agent can reasonably own the task

Requirements:

- LangGraph orchestration **still required**
- Single agent represented as one or more nodes
- Explicit state and stop conditions

Single-agent ≠ no orchestration
Orchestration is still mandatory.

🔒 Multi-Agent Capstone (Additional Mandatory Requirements)

If a project uses **more than one agent**, the following are **non-negotiable**:

1 Explicit Orchestration Layer

- A **central orchestrator** controls:
 - Agent selection
 - Task assignment

- Turn-taking
- Agents **must not** call each other directly

Orchestration owns *flow*. Agents own *work*.

2 Clear Agent Responsibilities

Each agent must have:

- A single, well-defined role
- Bounded scope
- Documented inputs and outputs

No “general” or “do-everything” agents.

3 Orchestration Patterns Used Intentionally

At least **one explicit pattern** must be implemented and documented:

- Supervisor / Worker
 - Plan → Execute → Validate
 - Fan-out → Fan-in
 - Escalation & fallback
-

4 State Isolation Rules

- Shared state vs agent-local state defined
- No accidental context leakage

- Orchestrator mediates all state transitions
-

5 Governance at the Orchestration Level

- MCP rules applied per agent
 - Tool permissions enforced by orchestrator
 - Step limits and stop conditions applied centrally
-

6 Explainability Requirement

The submission **must explain**:

- Why multiple agents were needed
- Why orchestration was chosen
- How decisions flow through the system

If it cannot be explained, it is not acceptable.

Explicitly Disallowed in Capstone

- ✗ Agents calling agents directly
- ✗ Unbounded agent loops
- ✗ Hidden shared memory
- ✗ “Swarm” or emergent behavior
- ✗ Undocumented decision logic

Any of the above = **automatic revision required**.

Capstone Evaluation Criteria (Updated)

Dimension	Evaluated On
Architecture	Explicit state, orchestration clarity
Control	Safe flow, stop conditions
Governance	MCP applied correctly
Retrieval	Grounded reasoning
Orchestration (Multi-Agent)	Mandatory & correct
Explainability	Can the system be defended
Delivery	Runnable, versioned, professional

Example Capstone Themes (Unchanged)

- Internal knowledge assistant
- Risk / compliance review agent
- Research synthesis system
- Decision-support workflow

Multi-agent use is **optional**, but if chosen, orchestration is **mandatory**.

Agentic AI Bootcamp — FINAL LOCKED CURRICULUM

Audience: Absolute Beginner → Agent-Ready → Enterprise-Capable

Philosophy: Understanding, reliability, governance, real-world design

Status:  FULLY LOCKED

Module 0 — Environment Setup for Agentic AI

Purpose: Remove friction before learning begins

Covers:

- Installing Python (supported versions)
- Installing VS Code
- Terminal basics (run, verify)
- Virtual environments (conceptual + setup)
- First successful Python run
- Installing packages (`pip`)

Explicitly NOT covered: Git, AI libraries, theory

 Outcome: Every learner can run Python locally with confidence.

Module A — Python for Agentic AI

Purpose: Build the control layer for agents (not general Python)

Core Sections

- Programming mindset for AI systems
- Running Python & scripts
- Variables as agent memory
- Arithmetic & logical operators (state changes & decisions)
- Strings (LLM-critical text handling)
- Lists, dictionaries, sets, tuples (agent state & constraints)
- Control flow (if/else as guardrails)
- Loops (evaluation & iteration)
- Functions (agent capabilities)
- Errors & safe failure
- Minimal AI project structure

✓ Outcome: Learners think in **state**, **flow**, **constraints**.

Module A+ — Type Annotations for Agent State (Pre-LangGraph Bridge)

Purpose: Prepare learners for explicit state & workflows

Covers:

- Why types matter in agent systems
- Basic annotations
- Typed lists & dictionaries
- **Optional** (incomplete state)

- **Union** (multiple valid outcomes)
- **TypedDict** (agent state blueprint)

Explicitly NOT covered: mypy, generics, protocols

✓ Outcome: Learners are **LangGraph-ready** without overload.

Module B — LLMs & Prompt Engineering

Purpose: Correct mental models before touching tools

Covers:

- What LLMs are / are not
- Strengths vs limitations
- Determinism vs variability
- Prompt anatomy (role, instruction, context, constraints, output)
- Instruction vs context
- Output control
- Prompt iteration (without guessing)
- Hallucinations & trust boundaries
- Prompting vs programming

B10 — Prompt Engineering Patterns (Explicit Naming)

- Zero-shot prompting
- Few-shot (in-context) prompting
- Role/persona assignment

- Contextual prompting
- Chain-of-Thought (used cautiously)

✓ Outcome: Learners design prompts **intentionally**, not experimentally.

Module C — Embeddings & Semantic Search

Purpose: Replace guessing with retrieval

Covers:

- Why LLMs guess
- What embeddings represent
- Semantic vs keyword search
- Embedding lifecycle
- Similarity search (conceptual)
- Minimal semantic search (in-memory)
- Strengths, limits, failure modes
- Role of embeddings in agent systems

Explicitly NOT covered: vector DBs, chunking depth

✓ Outcome: Learners understand **why retrieval matters**.

Module D — RAG (Retrieval → Reasoning → Grounded Output)

Purpose: Build reliable, grounded AI systems

Covers:

- Why RAG exists
- End-to-end RAG architecture
- Ingestion (conceptual)
- **Vector Databases (formal intro):**
 - FAISS (local)
 - Chroma (developer-friendly local DB)
 - Pinecone (managed, production)
- Retrieval as evidence selection
- Reasoning with guardrails
- RAG failure modes
- When RAG is the wrong tool

✅ Outcome: Learners can **design RAG correctly**, not just use it.

Module E — LangChain (LLM Application Orchestration)

Purpose: Implement systems cleanly and repeatedly

Covers:

- Why LangChain exists
- Core LangChain mental model
- Prompts in LangChain
- LLM invocation & output handling

Chains (Explicit Patterns)

- Sequential chains
- Conditional chains
- Parallel chains

Additional:

- RAG implementation via retrievers
- Tools as callable capabilities
- LangChain failure modes
- When NOT to use LangChain

✅ Outcome: Learners orchestrate workflows **without abstraction abuse**.

Module F — Agents in LLM Systems (Mental Models + Examples)

Purpose: Kill agent hype, establish correct thinking

Covers:

- Why agents exist
- What an agent actually is
- What is NOT an agent
- Agent components (state, policy, actions, loop)
- Levels of autonomy
- Conceptual agent examples
- Failure modes

- Guardrails & stop conditions
- Agents vs workflows

✓ Outcome: Learners understand **when agents are appropriate**.

Module G — LangGraph (State + Controlled Workflows)

Purpose: Make agents safe, inspectable, and enterprise-ready

Covers:

- Why LangGraph exists
- Core mental model (state, nodes, edges)
- Typed state definition
- Nodes as pure functions
- Conditional edges & loops
- Retries & stop conditions
- Debugging & explainability
- RAG inside LangGraph
- LangGraph vs LangChain decision boundary

✓ Outcome: Learners build **governed agent workflows**.

Module H — MCP (Context Boundaries + Governance)

Purpose: Control, trust, and audit agent systems

Covers:

- Dangers of unbounded context
- MCP as a governance mindset
- Types of context
- Context boundaries
- Permissions & capability control
- Governance rules
- Auditability & explainability
- MCP failure modes
- End-to-end governance walkthrough

✓ Outcome: Learners design **safe and defensible** systems.

Module I — APIs, Tools, Streamlit, Git/GitHub

Purpose: Ship, expose, and share real systems

Covers:

- APIs in agent systems
- Using APIs safely
- Tools as agent capabilities
- Streamlit for simple UIs
- Integrating LangChain/LangGraph with UI
- Git & GitHub fundamentals
- Clean project structure

- Delivery failure modes

✓ Outcome: Learners deliver **runnable, professional projects**.

Module J — Advanced RAG

Purpose: Move from basic RAG to production-grade retrieval

Covers:

- Why basic RAG fails
- Chunking strategies
- Metadata-aware retrieval
- Hybrid search
- Multi-stage retrieval
- Handling ambiguity
- RAG quality evaluation

✓ Outcome: Learners design **robust, scalable RAG systems**.

Module K — Multi-Agent Systems & Orchestration

Purpose: Power with control — no agent chaos

Covers:

- Why orchestration is mandatory
- What multi-agent really means
- Orchestration layer as control plane

- Agents as bounded specialists
- Global vs local state
- Orchestration patterns:
 - Supervisor/Worker
 - Plan → Execute → Validate
 - Fan-out/Fan-in
 - Escalation & fallback
- LangGraph as orchestrator
- MCP governance in orchestration
- Multi-agent failure modes
- When NOT to use multi-agent systems

✅ Outcome: Learners build **coordinated, governed multi-agent systems**.

Module L — Capstone Project (Enterprise-Grade Agentic AI)

MANDATORY FOR COMPLETION

All Capstones Must Include

- Typed state
- LangGraph orchestration
- RAG (basic or advanced)
- MCP governance

- Guardrails & stop conditions
- Streamlit UI or API
- GitHub repo with clean structure

Multi-Agent Capstones (Additional Mandatory Rules)

- Explicit orchestration layer
- No agent-to-agent direct calls
- Clear agent responsibilities
- At least one orchestration pattern
- Central governance enforcement
- Explainability of decisions

 Swarms, emergent behavior, uncontrolled autonomy **not allowed**

 Outcome: A **defensible, enterprise-grade Agentic AI system**